

PRACTICAL-9

Language recongnization using finite state automata and finite state machine

Context-Free Grammar (CFG) and Parsing

A Context-Free Grammar (CFG) is a formal way to describe the structure or syntax of languages. It consists of production rules that show how sentences are formed using terminals (words) and non-terminals (phrases).

Each rule defines how symbols can be replaced to generate valid strings in the language.

A parser like ChartParser in NLTK uses these rules to analyze whether a sentence follows the grammar and constructs a parse tree that shows the hierarchical relationship among words.

CFGs are widely used in natural language processing (NLP), compiler design, and syntax checking.

Example:

$S \rightarrow NP VP$ means a sentence is made up of a noun phrase followed by a verb phrase.

Code:

```
▶ import nltk
import spacy
from nltk import Nonterminal, CFG
from nltk.parse import ChartParser

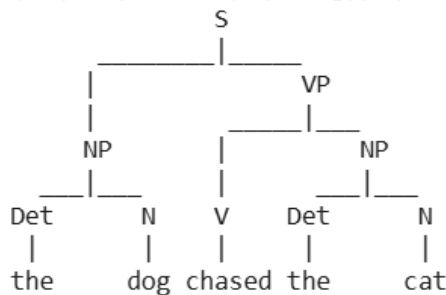
grammar = nltk.CFG.fromstring("""
S -> NP VP
NP -> Det N | 'I'
VP -> V NP | VP PP
PP -> P NP
Det -> 'the'
N -> 'cat' | 'dog'
V -> 'chased'
P -> 'on'
""")

rule = grammar.productions()
print(rule)
sentence = ['the', 'dog', 'chased', 'the', 'cat']
print(sentence)

parser = ChartParser(grammar)
for tree in parser.parse(sentence):
    print(tree)
    tree.pretty_print()
```

Output:

- [S -> NP VP, NP -> Det N, NP -> 'I', VP -> V NP, VP -> VP PP, PP -> P NP, |
['the', 'dog', 'chased', 'the', 'cat']
(S (NP (Det the) (N dog)) (VP (V chased) (NP (Det the) (N cat))))

**2. Finite Automata (DFA)**

A **Deterministic Finite Automaton (DFA)** is a mathematical model used to recognize **regular languages**. It consists of a finite number of **states**, an **input alphabet**, **transitions**, a **start state**, and one or more **final (accepting) states**.

The DFA reads an input string symbol by symbol and changes states according to its **transition function**. If the machine ends in a final state after reading the entire input, the string is **accepted**, otherwise **rejected**. In your code, the DFA accepts all strings that **start with the letter 'a'**.

Applications:

Lexical analysis, pattern recognition, and string validation in compilers and text processing.

Code:

```

# starting with a
vocabulary = ['a', 'b']
transitions = {
    ('q0', 'a'): 'q1',
    ('q1', 'a'): 'q1',
    ('q1', 'b'): 'q1',
    ('q0', 'b'): 'q2',
    ('q2', 'a'): 'q2',
    ('q2', 'b'): 'q2',
}
initial_state = 'q0'
final_states = {'q1'}

def analyze_string(string):
    current_state = initial_state
    for char in string:
        if (current_state, char) in transitions:
            current_state = transitions[(current_state, char)]
        else:
            return False
    return current_state in final_states

analyze_string("aaabb")

```

... True

Mealy Machine

A Mealy Machine is a type of finite state machine where the output depends on both the current state and the input symbol.

Each transition is labeled with both an output and the next state — (next_state, output).

It is often used in digital circuit design, sequence detectors, and controllers.

In your implementation, the Mealy machine processes a binary string and produces outputs at each step. The final output represents the remainder when the binary number is divided by 3 (i.e., computing %3).

Key difference from Moore Machine:

- Mealy: Output depends on state + input
- Moore: Output depends only on state

Code:



Mealy machine

```
transitions = {
    ('S0', '0'): ('S0', '0'),
    ('S0', '1'): ('S1', '1'),
    ('S1', '1'): ('S0', '0'),
    ('S1', '0'): ('S2', '2'),
    ('S2', '1'): ('S2', '2'),
    ('S2', '0'): ('S1', '1')
}

def mealy(string):
    string=string.lower()
    current_state='S0'
    path=[current_state]
    output=[]

    for char in string:
        if(char not in ['0','1']):
            print("invalid string")
        transition=(current_state,char)
        if transition in transitions:
            (current_state,op) = transitions[transition]
            path.append(current_state)
            output.append(op)
            print(f" character: {char} moved to state {current_state} ouptut : {op}")
```

```
print(f" output : {output}")  
print(f"{string}%3 = {output[-1]}")
```

```
string= '100110'  
print(f"Finding the %3 for binary string {string}")  
mealy(string)
```

```
Finding the %3 for binary string 100110  
character: 1 moved to state S1 ouptut : 1  
character: 0 moved to state S2 ouptut : 2  
character: 0 moved to state S1 ouptut : 1  
character: 1 moved to state S0 ouptut : 0  
character: 1 moved to state S1 ouptut : 1  
character: 0 moved to state S2 ouptut : 2  
output : ['1', '2', '1', '0', '1', '2']  
100110%3 = 2
```